

使用 Cloud Run、Video Intelligence API 和 Vertex AI，建立影片場景影像說明服務

來源：<https://codelabs.developers.google.com/codelabs/how-to-create-cloud-run-service-for-describing-scenes-in-a-video?hl=zh-tw>

步驟數：10

瞭解如何使用 Vertex AI 和 Video Intelligence API 建立 Cloud Run 服務來說明影片的各個場景中的圖片。

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [啟用 API 並設定環境變數](#)
4. [建立 Cloud Storage 值區](#)
5. [建立 Node.js 應用程式](#)
6. [建立服務帳戶](#)
7. [部署 Cloud Run 服務](#)
8. [呼叫 Cloud Run 服務](#)
9. [恭喜！](#)
10. [清除所用資源](#)

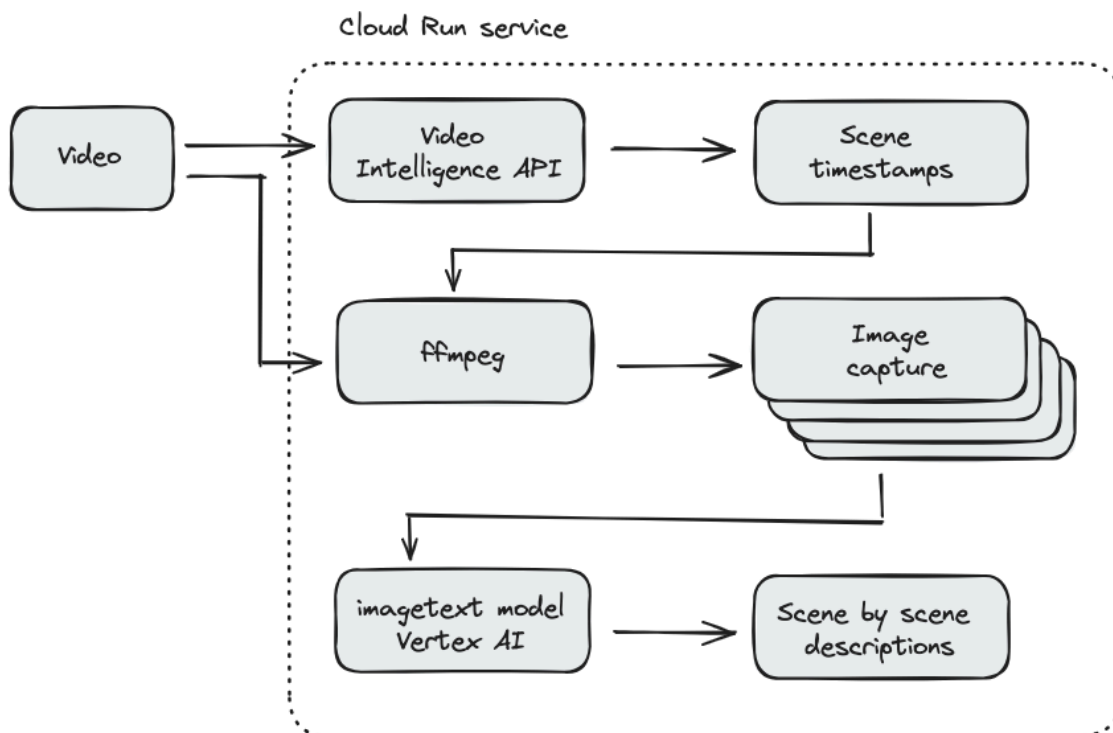
1. 簡介

總覽

在本程式碼研究室中，您將建立以 Node.js 編寫的 Cloud Run 服務，提供影片中每個場景的視覺描述。首先，服務會使用 Video Intelligence API 偵測場景變更的時間戳記。接著，服務會使用名為 ffmpeg 的第三方二進位檔，擷取每個場景變更時間戳記的螢幕截圖。最後，我們使用 Vertex AI 圖像說明生成功能，提供螢幕截圖的視覺描述。

本程式碼實驗室也示範如何在 Cloud Run 服務中使用 ffmpeg，擷取特定時間戳記的影片畫面。由於 ffmpeg 必須獨立安裝，本程式碼實驗室會說明如何建立 Dockerfile，以便在 Cloud Run 服務中安裝 ffmpeg。

以下是 Cloud Run 服務的運作方式插圖：



課程內容

- 如何使用 Dockerfile 建立容器映像檔，安裝第三方二進位檔
- 如何遵循最小權限原則，為 Cloud Run 服務建立服務帳戶，以便呼叫其他 Google Cloud 服務
- 如何從 Cloud Run 服務使用 Video Intelligence 用戶端程式庫
- 如何呼叫 Google API，從 Vertex AI 取得每個場景的影像說明

步驟 2 · 約 0 分鐘

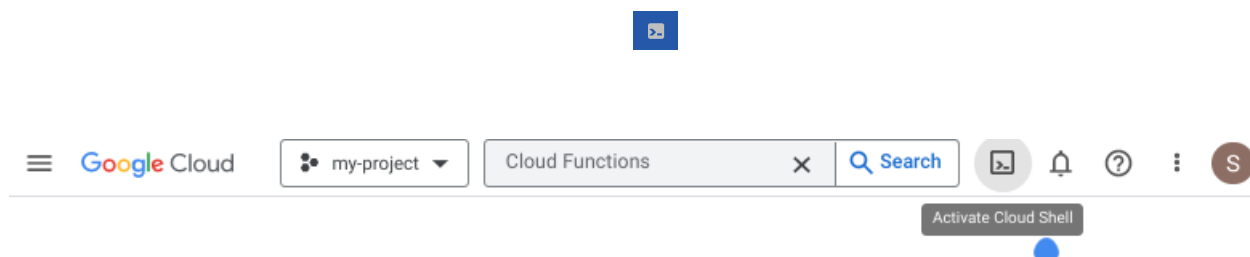
2. 設定和需求

必要條件

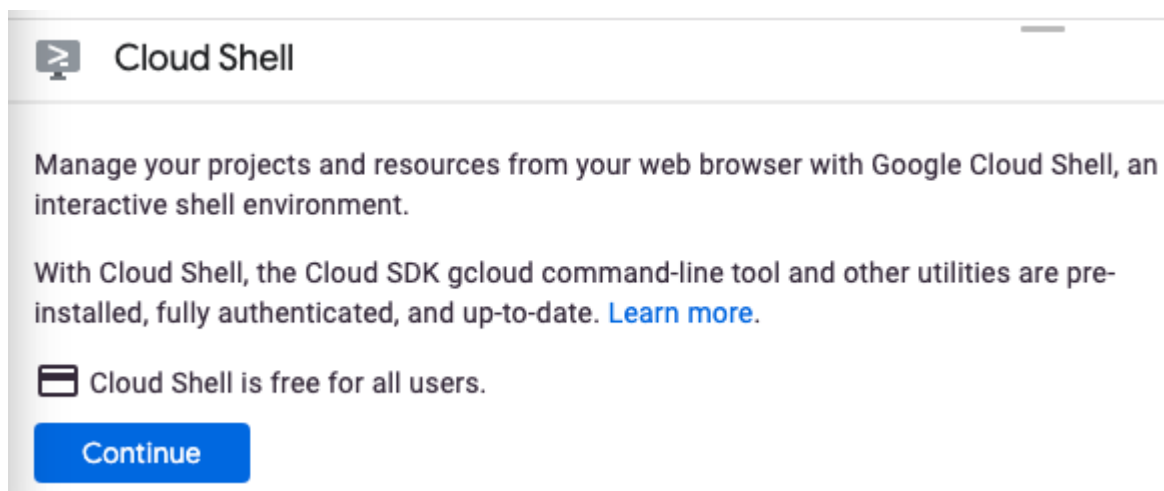
- 您已登入 Cloud Console。
- 您先前已部署 Cloud Run 服務。舉例來說，您可以按照[從原始碼部署網路服務的快速入門導覽課程](#)，開始使用 Cloud Run。

啟用 Cloud Shell

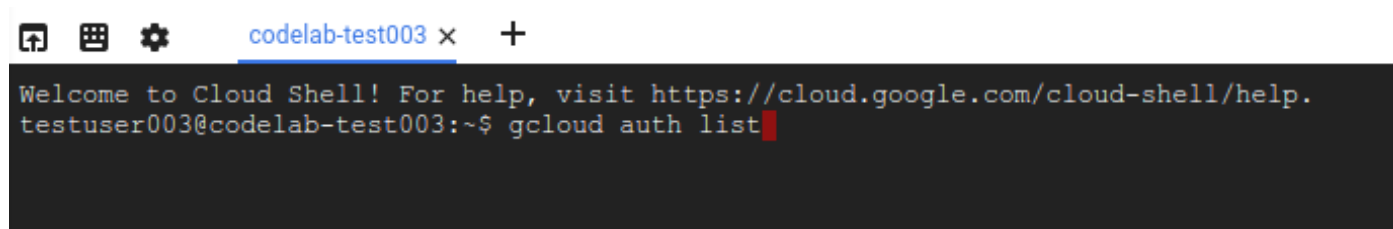
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出



```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 0 分鐘

3. 啟用 API 並設定環境變數

開始進行本程式碼研究室之前，請先啟用數個 API。本程式碼研究室需要使用下列 API。執行下列指令即可啟用這些 API：

```
gcloud services enable run.googleapis.com \  
  storage.googleapis.com \  
  cloudbuild.googleapis.com \  
  videointelligence.googleapis.com \  
  aiplatform.googleapis.com
```

接著，您可以設定本程式碼研究室全程都會用到的環境變數。

```
REGION=<YOUR-REGION>  
PROJECT_ID=<YOUR-PROJECT-ID>  
  
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --  
format='value(projectNumber)')  
SERVICE_NAME=video-describer  
export BUCKET_ID=$PROJECT_ID-video-describer
```

步驟 4 · 約 0 分鐘

4. 建立 Cloud Storage 值區

使用下列指令建立 Cloud Storage bucket，以便上傳影片供 Cloud Run 服務處理：

```
gsutil mb -l us-central1 gs://$BUCKET_ID/
```

[選用] 您可以下載這個[影片樣本](#)。

```
gsutil cp gs://cloud-samples-data/video/visionapi.mp4 testvideo.mp4
```

現在請將影片檔案上傳至儲存空間 bucket。

```
FILENAME=<YOUR-VIDEO-FILENAME>  
gsutil cp $FILENAME gs://$BUCKET_ID
```

步驟 5 · 約 0 分鐘

5. 建立 Node.js 應用程式

首先，請建立原始碼的目錄，然後 cd 到該目錄。

```
mkdir video-describer && cd $_
```

接著，建立 package.json 檔案，並加入以下內容：

```
{
  "name": "video-describer",
  "version": "1.0.0",
  "private": true,
  "description": "describes the image in every scene for a given video",
  "main": "index.js",
  "author": "Google LLC",
  "license": "Apache-2.0",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "@google-cloud/storage": "^7.7.0",
    "@google-cloud/video-intelligence": "^5.0.1",
    "axios": "^1.6.2",
    "express": "^4.18.2",
    "fluent-ffmpeg": "^2.1.2",
    "google-auth-library": "^9.4.1"
  }
}
```

這個應用程式包含多個來源檔案，可提高可讀性。首先，請建立 index.js 來源檔案，並加入下列內容。這個檔案包含服務的進入點，以及應用程式的主要邏輯。


```

const { captureImages } = require('./imageCapture.js');
const { detectSceneChanges } = require('./sceneDetector.js');
const transcribeScene = require('./imageDescriber.js');
const { Storage } = require('@google-cloud/storage');
const fs = require('fs').promises;
const path = require('path');
const express = require('express');
const app = express();

const bucketName = process.env.BUCKET_ID;

const port = parseInt(process.env.PORT) || 8080;
app.listen(port, () => {
  console.log(`video describer service ready: listening on port ${port}`);
});

// entry point for the service
app.get('/', async (req, res) => {

  try {

    // download the requested video from Cloud Storage
    let videoFilename = req.query.filename;
    console.log("processing file: " + videoFilename);

    // download the file to locally to the Cloud Run instance
    let localFilename = await downloadVideoFile(videoFilename);

    // detect all the scenes in the video & save timestamps to an array
    let timestamps = await detectSceneChanges(localFilename);
    console.log("Detected scene changes at the following timestamps: ", timestamps);

    // create an image of each scene change
    // and save to a local directory called "output"
    await captureImages(localFilename, timestamps);

    // get an access token for the Service Account to call the Google APIs
    let accessToken = await transcribeScene.getAccessToken();
    console.log("got an access token");

    let imageBaseName = path.parse(localFilename).name;

    // the data structure for storing the scene description and timestamp
    // e.g. an array of json objects {timestamp: 1, description: "..."}, etc.
    let scenes = []

    // for each timestamp, send the image to Vertex AI
    console.log("getting Vertex AI description all the timestamps");
  }
}

```

```

    scenes = await Promise.all(
      timestamps.map(async (timestamp) => {

        let filepath = path.join("./output", imageBaseName + "-" + timestamp +
".png");

        // get the base64 encoded image
        const encodedFile = await fs.readFile(filepath, 'base64');

        // send each screenshot to Vertex AI for description
        let description = await transcribeScene.transcribeScene(accessToken,
encodedFile)

        return { timestamp: timestamp, description: description };
      }));

    console.log("finished collecting all the scenes");
    //console.log(scenes);

    return res.json(scenes);

  } catch (error) {

    //return an error
    console.log("received error: ", error);
    return res.status(500).json("an internal error occurred");
  }
});

async function downloadVideoFile(videoFilename) {
  // Creates a client
  const storage = new Storage();

  // keep same name locally
  let localFilename = videoFilename;

  const options = {
    destination: localFilename
  };

  // Download the file
  await storage.bucket(bucketName).file(videoFilename).download(options);

  console.log(
    `gs://${bucketName}/${videoFilename} downloaded locally to ${localFilename}.`
  );
};

```

```
    return localFilename;
}
```

接著，建立 `sceneDetector.js` 檔案，並加入以下內容。這個檔案會使用 Video Intelligence API，偵測影片中的場景變化。

```

const fs = require('fs');
const util = require('util');
const readFile = util.promisify(fs.readFile);
const ffmpeg = require('fluent-ffmpeg');

const Video = require('@google-cloud/video-intelligence');
const client = new Video.VideoIntelligenceServiceClient();

module.exports = {
  detectSceneChanges: async function (downloadedFile) {

    // Reads a local video file and converts it to base64
    const file = await readFile(downloadFile);
    const inputContent = file.toString('base64');

    // setup request for shot change detection
    const videoContext = {
      speechTranscriptionConfig: {
        languageCode: 'en-US',
        enableAutomaticPunctuation: true,
      },
    };

    const request = {
      inputContent: inputContent,
      features: ['SHOT_CHANGE_DETECTION'],
    };

    // Detects camera shot changes
    const [operation] = await client.annotateVideo(request);
    console.log('Shot (scene) detection in progress...');
    const [operationResult] = await operation.promise();

    // Gets shot changes
    const shotChanges = operationResult.annotationResults[0].shotAnnotations;

    console.log("Shot (scene) changes detected: " + shotChanges.length);

    // data structure to be returned
    let sceneChanges = [];

    // for the initial scene
    sceneChanges.push(1);

    // if only one scene, keep at 1 second
    if (shotChanges.length === 1) {
      return sceneChanges;
    }
  }
};

```

```

    // get length of video
    const videoLength = await getVideoLength(downloadedFile);

    shotChanges.forEach((shot, shotIndex) => {
        if (shot.endTimeOffset === undefined) {
            shot.endTimeOffset = {};
        }
        if (shot.endTimeOffset.seconds === undefined) {
            shot.endTimeOffset.seconds = 0;
        }
        if (shot.endTimeOffset.nanos === undefined) {
            shot.endTimeOffset.nanos = 0;
        }

        // convert to a number
        let currentTimestampSecond = Number(shot.endTimeOffset.seconds);

        let sceneChangeTime = 0;
        // double-check no scenes were detected within the last second
        if (currentTimestampSecond + 1 > videoLength) {
            sceneChangeTime = currentTimestampSecond;
        } else {
            // otherwise, for simplicity, just round up to the next second
            sceneChangeTime = currentTimestampSecond + 1;
        }

        sceneChanges.push(sceneChangeTime);
    });

    return sceneChanges;
}

}

async function getVideoLength(localFile) {
    let getLength = util.promisify(ffmpeg.ffprobe);
    let length = await getLength(localFile);

    console.log("video length: ", length.format.duration);
    return length.format.duration;
}

```

現在，請建立名為 `imageCapture.js` 的檔案，並加入下列內容。這個檔案會使用節點套件 [fluent-ffmpeg](#)，從節點應用程式內執行 `ffmpeg` 指令。

```

const ffmpeg = require('fluent-ffmpeg');
const path = require('path');
const util = require('util');

module.exports = {
  captureImages: async function (localFile, scenes) {

    let imageBaseName = path.parse(localFile).name;

    try {
      for (scene of scenes) {
        console.log("creating screenshot for scene: ", + scene);
        await createScreenshot(localFile, imageBaseName, scene);
      }

    } catch (error) {
      console.log("error gathering screenshots: ", error);
    }

    console.log("finished gathering the screenshots");
  }
}

async function createScreenshot(localFile, imageBaseName, scene) {
  return new Promise((resolve, reject) => {
    ffmpeg(localFile)
      .screenshots({
        timestamps: [scene],
        filename: `${imageBaseName}-${scene}.png`,
        folder: 'output',
        size: '320x240'
      })
      .on("error", () => {
        console.log("Failed to create scene for timestamp: " + scene);
        return reject('Failed to create scene for timestamp: ' + scene);
      })
      .on("end", () => {
        return resolve();
      });
  })
}

```

最後，建立名為 `imageDescriber.js` 的檔案，並加入以下內容。這個檔案會使用 Vertex AI 取得每個場景圖片的視覺說明。

```

const axios = require("axios");
const { GoogleAuth } = require('google-auth-library');

const auth = new GoogleAuth({
  scopes: 'https://www.googleapis.com/auth/cloud-platform'
});

module.exports = {
  getAccessToken: async function () {

    return await auth.getAccessToken();
  },

  transcribeScene: async function(token, encodedFile) {

    let projectId = await auth.getProjectId();

    let config = {
      headers: {
        'Authorization': 'Bearer ' + token,
        'Content-Type': 'application/json; charset=utf-8'
      }
    }

    const json = {
      "instances": [
        {
          "image": {
            "bytesBase64Encoded": encodedFile
          }
        }
      ],
      "parameters": {
        "sampleCount": 1,
        "language": "en"
      }
    }

    let response = await axios.post('https://us-central1-aiplatform.googleapis.com/v1/projects/' + projectId + '/locations/us-central1/publishers/google/models/imagetext:predict', json, config);

    return response.data.predictions[0];
  }
}

```


建立 Dockerfile 和 .dockerignore 檔案

由於這項服務使用 ffmpeg，您需要建立安裝 ffmpeg 的 Dockerfile。

建立名為 `Dockerfile` 的檔案，並加入以下內容：

```
# Copyright 2020 Google, LLC.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

# Use the official lightweight Node.js image.
# https://hub.docker.com/_/node
FROM node:20.10.0-slim

# Create and change to the app directory.
WORKDIR /usr/src/app

RUN apt-get update && apt-get install -y ffmpeg

# Copy application dependency manifests to the container image.
# A wildcard is used to ensure both package.json AND package-lock.json are copied.
# Copying this separately prevents re-running npm install on every code change.
COPY package*.json ./

# Install dependencies.
# If you add a package-lock.json speed your build by switching to 'npm ci'.
# RUN npm ci --only=production
RUN npm install --production

# Copy local code to the container image.
COPY . .

# Run the web service on container startup.
CMD [ "npm", "start" ]
```

並建立名為 `.dockerignore` 的檔案，忽略特定檔案的容器化作業。

```
Dockerfile
.dockerignore
node_modules
npm-debug.log
```

6. 建立服務帳戶

您將建立服務帳戶，供 Cloud Run 服務存取 Cloud Storage、Vertex AI 和 Video Intelligence API。

```
SERVICE_ACCOUNT="cloud-run-video-description"
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com

gcloud iam service-accounts create $SERVICE_ACCOUNT \
  --display-name="Cloud Run Video Scene Image Describer service account"

# to view & download storage bucket objects
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \
  --role=roles/storage.objectViewer

# to call the Vertex AI imagetext model
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \
  --role=roles/aiplatform.user
```

步驟 7 · 約 0 分鐘

7. 部署 Cloud Run 服務

您現在可以使用以來源為基礎的部署作業，自動將 Cloud Run 服務容器化。

注意：Cloud Run 服務的預設處理時間為 60 秒。本程式碼研究室使用 5 分鐘的逾時時間，因為建議的測試影片長度為 2 分鐘。如果使用時間長度較長的影片，可能需要修改時間。

```
gcloud run deploy $SERVICE_NAME \  
  --region=$REGION \  
  --set-env-vars BUCKET_ID=$BUCKET_ID \  
  --no-allow-unauthenticated \  
  --service-account $SERVICE_ACCOUNT_ADDRESS \  
  --timeout=5m \  
  --source=.
```

部署完成後，請將服務網址儲存至環境變數。

```
SERVICE_URL=$(gcloud run services describe $SERVICE_NAME --platform managed --region  
$REGION --format 'value(status.url)')
```

步驟 8 · 約 0 分鐘

8. 呼叫 Cloud Run 服務

現在，只要提供您上傳至 Cloud Storage 的影片名稱，即可呼叫服務。

```
curl -X GET -H "Authorization: Bearer $(gcloud auth print-identity-token)"  
${SERVICE_URL}?filename=${FILENAME}
```

結果應類似下方的輸出範例：

```
[{"timestamp":1,"description":"an aerial view of a city with a bridge in the  
background"}, {"timestamp":7,"description":"a man in a blue shirt sits in front of  
shelves of donuts"}, {"timestamp":11,"description":"a black and white photo of people  
working in a bakery"}, {"timestamp":12,"description":"a black and white photo of a man  
and woman working in a bakery"}]
```

步驟 9 · 約 0 分鐘

9. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Video Intelligence API](#)、[Cloud Run](#) 和 [Vertex AI 圖像說明生成](#) 的說明文件。

涵蓋內容

- 如何使用 Dockerfile 建立容器映像檔，安裝第三方二進位檔
 - 如何遵循最小權限原則，為 Cloud Run 服務建立服務帳戶，以便呼叫其他 Google Cloud 服務
 - 如何從 Cloud Run 服務使用 Video Intelligence 用戶端程式庫
 - 如何呼叫 Google API，從 Vertex AI 取得每個場景的影像說明
-

10. 清除所用資源

為避免產生意外費用 (例如，如果這個 Cloud Run 服務的叫用次數不慎超過[免費層級的每月 Cloud Run 叫用次數配額](#))，您可以刪除 Cloud Run 服務，或刪除在步驟 2 中建立的專案。

如要刪除 Cloud Run 服務，請前往 <https://console.cloud.google.com/run/> 的 Cloud Run Cloud 控制台，然後刪除 `video-describer` 函式 (如果您使用其他名稱，請刪除 `$SERVICE_NAME`)。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
